



Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Santa Fe

**Modelación de sistemas multiagentes con gráficas Computacionales
(Gpo 301)**

Nombre del profesores:

Octavio Navarro Hinojosa

Gilberto Echeverría Furío

Actividad:

Documentación Reto Movilidad Urbana

Alumno:

Nicolas Quintana Kluchnik |

A01785655

Eder Jezrael Cantero Moreno|

A01785888

Fecha:

4 de Diciembre del 2025

Introducción.....	2
Problema:.....	3
Necesidad:.....	3
Meta:.....	3
Propuesta:.....	3
Diseño de los agentes.....	3
Objetivo de Cada Agente:.....	3
Capacidad Efectora:.....	3
Percepción:.....	4
Proactividad:.....	4
Arquitectura de Subsunción.....	5
Características del Ambiente.....	5
Datos Recolectados.....	5
Consumo de datos:.....	6
Interpretación y posibles derivados:.....	6

Introducción

La movilidad urbana es un componente crítico para la competitividad económica y la calidad de vida. En México, los kilómetros-auto recorridos se triplicaron entre 1990 y 2010, acompañados de externalidades negativas (accidentes, smog, congestión). Este proyecto aborda el problema mediante una simulación multiagente que permite experimentar con configuraciones de infraestructura y políticas sencillas de control de flujo, conectando decisiones locales con métricas de desempeño agregadas.

Problema

Congestión vehicular derivada de alta motorización y gestión limitada del flujo en entornos urbanos.

Necesidad

Evaluar, en un entorno controlado, cómo reglas locales (sentido de calles, semáforos, generación de vehículos) influyen en la congestión y el tiempo de llegada a destinos.

Meta

Ofrecer una base experimental reproducible que conecte diseño vial con indicadores de throughput y acumulación, y que pueda extenderse hacia políticas adaptativas.

Propuesta

Se propone implementar un sistema basado en agentes reactivos con capacidades de planificación utilizando el framework Mesa para simulación multiagente y un algoritmo para optimizar el camino de los agentes. Encima de esto, se utilizara el sistema de servidores Flask y los módulos de Node para crear una simulación de multiagentes en 3D para visualizar la evolución del flujo para inspección cualitativa y comunicación de resultados adquiridos. Proveer configurabilidad en las áreas de los mapas, semillas/seeds, intervalo y volumen de spawn para comparar situaciones. Finalmente, implementar métricas básicas tal como la cantidad de coches creados, activos y llegados a su destino para ver el sistema en trabajo.

Diseño de los agentes

Objetivo de Cada Agente:

Coche(Car)	Traffic Light	Calle(Road)	Destination	Obstacle
Llegar al destino predeterminado cuando se crea	Modular flujo en las intersecciones	Infraestructura estática, define la dirección de flujo con "<, >, v, ^"	Indica las metas	Actúan como escenografía

Capacidad Efectora:

Los agentes coches tienen la siguiente capacitación

Acción	Descripción	Efecto
Movimiento	Desplazarse a una celda adyacente mientras que respete los parámetros	Está a una celda más cercana de su destino
Recalcular ruta	Si la ruta predeterminada hay	Busca una ruta más rápida para

	congestión, se calcula una ruta diferente para llegar al destino	llegar a su destino
Llegar al destino	Al llegar a su destino, se mete a la celda del destino	Desaparece del entorno virtual
Detención	Al llegar a un semáforo en luz rojo, se debe de respetar la ley	Parar y esperar hasta que el semáforo se vuelva verde

Percepción:

Elemento Percibido	Como lo percibe	Información obtenida
Límites del mapa	Válida coordenadas dentro de width/height antes de moverse (Car.can_move_to)	Si la celda está dentro de la grilla o no es accesible
Ocupación de celda destino	Revisa agentes en la celda objetivo: bloquea Obstacle, Gradas y otros Car; permite Destination (Car.can_move_to)	Si puede entrar, si hay destino, o si debe esperar/recalcular
Estado de semáforo	Lee Traffic_Light.state en la celda destino (Car.can_move_to)	Si el semáforo está verde o rojo para decidir avanzar o esperar
Dirección vial	Consulta dirección del Road en celda actual/siguiente (CityModel.get_road_direction , Car.can_move_to)	Sentido permitido; penaliza o bloquea contra-flujo
Navegabilidad de celda	Evalúa si hay Road, Traffic_Light o Destination y ausencia de obstáculos (CityModel.is_valid_road)	Determina si la celda forma parte de la red transitable
Congestión local	Calcula peso adicional si hay Car en la celda durante A* (CityModel.get_cell_weight)	Costo mayor en celdas congestionadas para desviar ruta
Distancia al destino	Usa distancia Manhattan (CityModel.heuristic) y vecinos explorables (Car.find_alternative_move)	Preferencia de ruta y selección de alternativas más cercanas
Paso del tiempo (semáforo)	Contador de pasos interno (Traffic_Light.step)	Cuándo alternar de rojo a verde y viceversa

Proactividad:

El coche es un agente proactivo que toma estas iniciativas inteligentes:

- Búsqueda del camino más corto a su destino: Al usar el algoritmo A* (A star), encuentra el camino más corto y menos congestionado para llegar a su destino
- Decisiones Autónomas: Elige su próximo movimiento basado en su estado actual y su alrededor
- Respeto de leyes: Con un diccionario implementado, el coche respeta las leyes/reglas/parámetros que se le da.

Métricas de Desempeño:

Arquitectura de Subsunción

Las decisiones de cada coche se estructuran en capas de prioridad decreciente, integradas en el step de Car:

- Seguridad y factibilidad (Car.can_move_to):
 - Primera barrera; descarta movimientos que salgan del mapa, invadan celdas con obstáculos/gradas/otros autos o entren a un semáforo en rojo. Si falla, el coche espera y aumenta los contadores de bloqueo.
- Cumplimiento vial (CityModel.is_move_allowed_by_road):
 - Segunda barrera; usa la dirección del Road en la celda actual para filtrar movimientos. Contra-flujo puro se bloquea; contra-flujo parcial se permite con penalización en planificación.
- Planificación global (CityModel.find_path):
 - A* con heurística Manhattan sobre celdas navegables. Penaliza celdas con autos y con dirección opuesta; permite movimientos laterales (cambio de carril) con mayor costo. Si no hay ruta, se reintenta tras rebasar los umbrales de bloqueo.
- Recuperación local (Car.find_alternative_move):
 - Cuando el path existe pero está bloqueado, explora vecinos compatibles con la dirección actual y elige el más cercano al destino; si no hay vecinos válidos, fuerza un nuevo A*.
- Gestión poblacional y terminación (CityModel.spawn_new_cars):
 - Regula generación periódica respetando disponibilidad de esquinas; cuenta intentos fallidos para detener la simulación y limpiar autos activos si la red queda saturada.
- Telemetría (CityModel.step):
 - Cada 5 pasos se publica estado agregado; esta capa no modifica el movimiento pero condiciona el ritmo de monitoreo.

Características del Ambiente

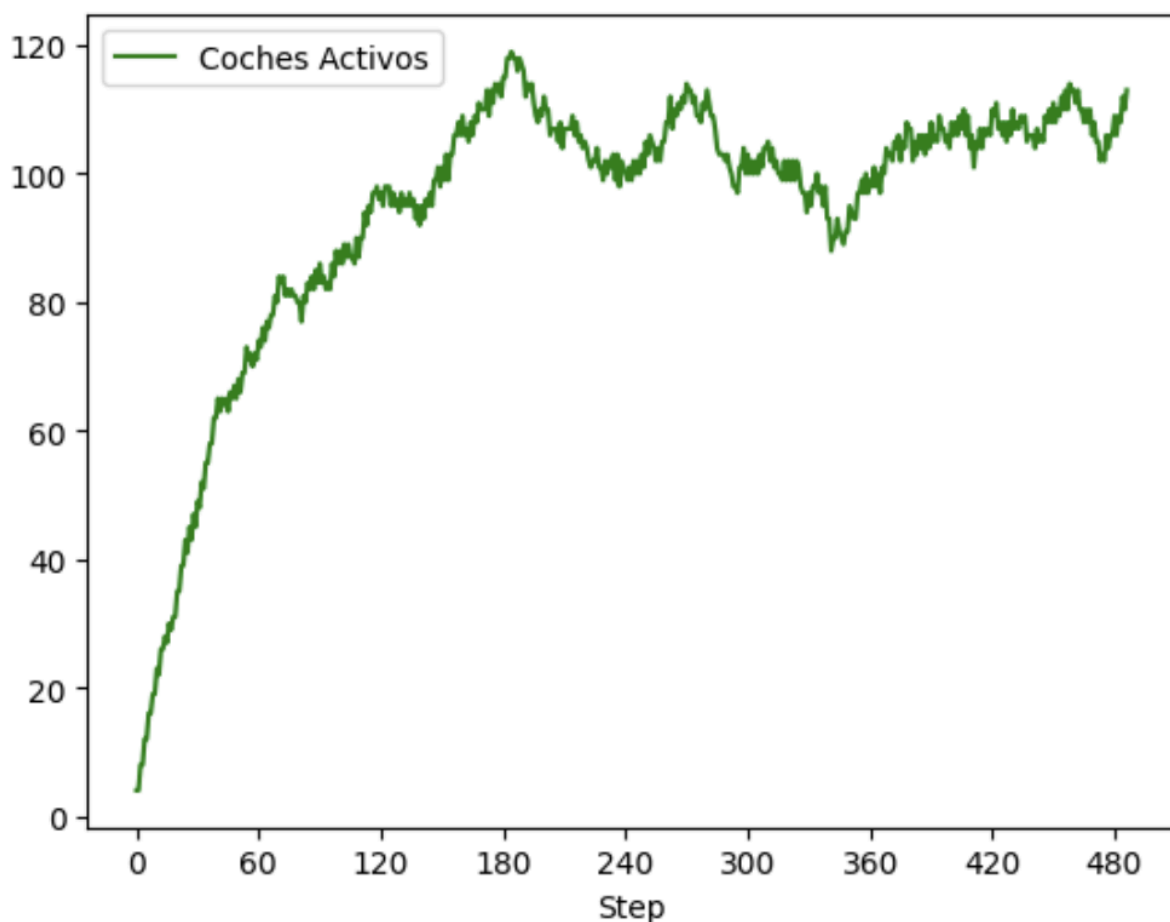
Característica	Descripción
Tipo de grid	OrthogonalMooreGrid (8 direcciones)

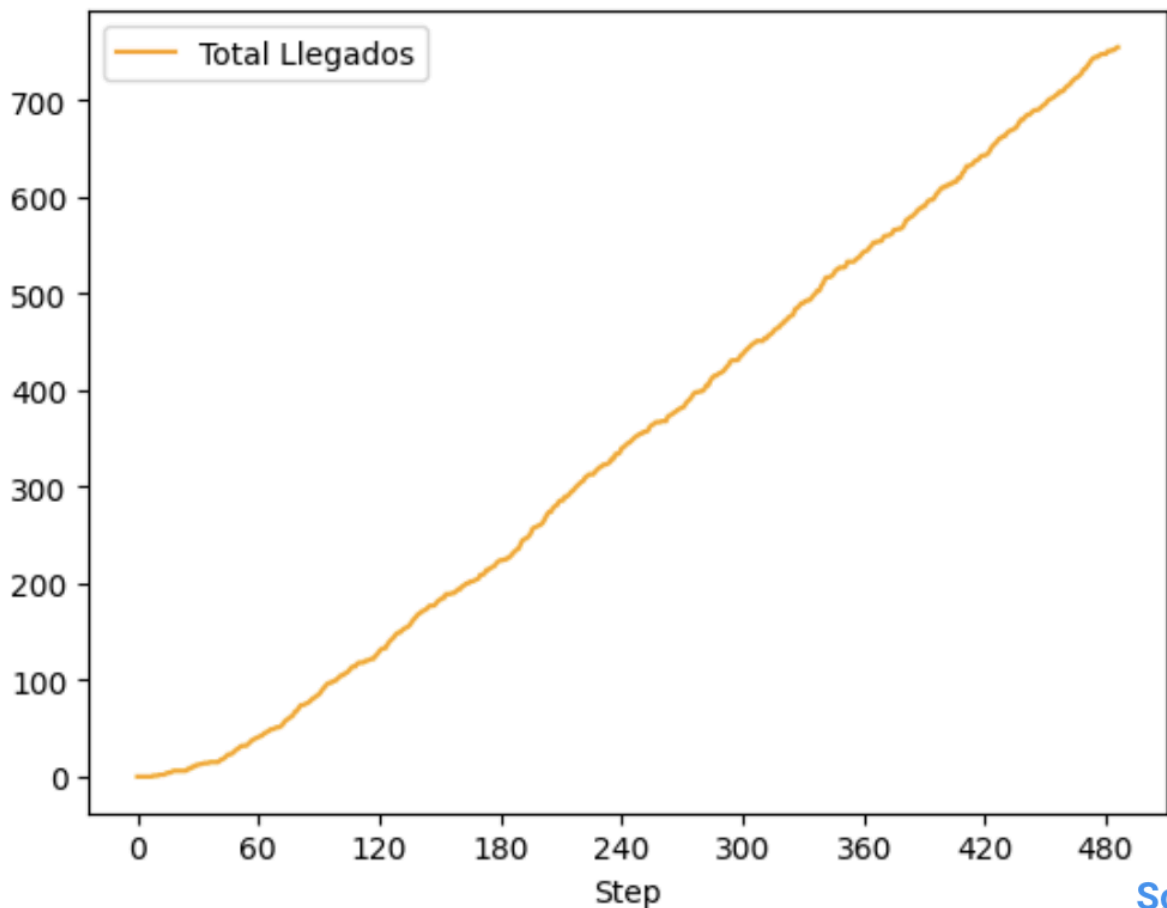
Tamaño	36x35 pero es personalizable el tamaño
Secuencial	Las decisiones afectan los estados futuros
Observabilidad	Parcialmente accesible ya que solo pueden observar las celdas que el algoritmo ha detectado
Determinístico	El estado siguiente completamente depende del estado actual
Dinamicidad	Entorno se mantiene constante

Datos Recolectados

La recolección de datos fue muy simple ya que teníamos la base del código de tareas previas. Los datos que fueron recolectados de forma automática fueron:

- Total Creados (DataCollector): incrementa al instanciar cada coche (CityModel.total_cars_created).
- Coches Activos (DataCollector): conteo de agentes Car vivos en cada paso.
- Total Llegados (DataCollector): acumulado al eliminar coches que llegaron a su destino (CityModel.total_cars_arrived).
- current_cars y total_arrived (API externa): publicados cada 5 pasos junto con step. (Datos previos pasan por la API para el concurso)





Lo que podemos ver en las gráficas son el total de coches activos por step en el entorno y la cantidad de coches que han llegado al destino. Podemos ver que al principio la cantidad de coches en el entorno sube de forma exponencial, pero ya cuando llegamos por el step 180, empezamos a ver que baja, esto nos dice que hay muchos coches que estaban llegando a sus destinos, esto es gracias a las intersecciones liberándose y dejando que todo fluya. Otra cosa que nos dice la gráfica es que al principio de la simulación, no había coches llegando a sus destinos, esto es gracias a que los destinos estaban muy lejos o llegaban a las intersecciones y había tráfico.

Consumo de datos:

- La UI Solara (Mesa) muestra tres gráficas de series de tiempo con los valores del DataCollector.
- Un cliente externo puede almacenar las publicaciones periódicas para análisis offline.

Interpretación y posibles derivados:

- $\text{Throughput} \approx \text{Total Llegados} / \text{pasos}$.
- Tasa de rechazo implícita: si Total Creados se estanca mientras spawn_interval sigue activo, hay saturación.
- Razón activos/creados: proxy de congestión; una caída rápida indica buena evacuación.

- Serie de Coches Activos: Una tendencia creciente sugiere acumulación; las oscilaciones reflejan ciclos de semáforos.

Áreas de Mejora

Ya que se hizo el proyecto, se encontraron cosas que pueden hacer este entorno más realista, se pueden agregar más tipos de coches, por ejemplo buses, ambulancias, policías, y motos para nombrar. Cada uno tendría su propia set de reglas, las ambulancias tendrán prioridad en las intersecciones para llegar a sus destinos lo más rápido posible, los policías pueden parar a los coches que no están respetando las reglas. Otra cosa que también se puede implementar es el uso de peatones, dándole a los coches otra cosa que deberían tomar en cuenta, haciéndolos más inteligentes y reactivos. Para ayudar con todo lo que se estaría agregando al entorno, ayudaría mucho hacer el mapa más grande e igual agregar otro carril a las calles, haciéndolas de 3 carriles para ayudar con el incremento de los agentes que estarían en el entorno.

Conclusiones

Este sistema implementa coches autónomos inteligentes que navegan una ciudad gracias al algoritmo A* y una toma de decisiones jerarquizada: El agente es más capaz de percibir su entorno local, priorizar objetivos (Que camino tomar para llegar a su destino) y actuando de manera autónoma para minimizar el tiempo que están en el entorno desde su “spawn” a su destino.